Documentation Technique — Outil de Transformation EJB-to-REST

Compleo | Document technique interne | Version 2.0 | Mars 2026

Table des matières

1. Introduction et objectifs
2. Vue d'ensemble du pipeline
3. Architecture technique
4. Étape 1 — Upload du projet EJB
5. Étape 2 — Analyse statique du code EJB
6. Étape 3 — Génération du projet REST
7. Étape 4 — Amélioration par la moulinette IA (SmartCodeEnhancer)
8. Étape 5 — Résultats et export
9. Règles de gestion (G1-G14)
10. Catalogue complet des 77 règles SmartCodeEnhancer
11. Transformation des Message-Driven Beans (MDB)
12. Extraction et génération des types métier (Axe 1)
13. Commentaires de traçabilité (Axe 2)
14. Transformation des annotations de sécurité (Axe 3)
15. Structure du projet généré
16. Mapping EJB vers REST — Correspondances
17. Score de qualité et métriques
18. Annexes techniques

1. Introduction et objectifs

L'outil **EJB-to-REST** est une application web développée en Java (Spring Boot) qui automatise la transformation de projets EJB legacy en API REST modernes basées sur Spring Boot 3.2.5. Cet outil a été conçu pour répondre au besoin de modernisation des applications bancaires existantes, en préservant la logique métier encapsulée dans les EJB tout en exposant cette logique via des endpoints REST conformes aux standards actuels.

Le processus de transformation s'articule autour de **cinq étapes séquentielles** : l'import du projet source, l'analyse statique du code EJB, la génération du code REST, l'amélioration automatique par un moteur de règles interne (la « moulinette IA »), et enfin l'export du projet généré. Chaque étape est orchestrée par une interface web intuitive qui guide l'utilisateur tout au long du processus.

Depuis la version 2.0, l'outil prend en charge la transformation complète des **Message-Driven Beans** (MDB) en composants Spring event-driven, l'extraction et la recopie des **enums métier**, **exceptions custom** et **validateurs custom** (annotations `@Constraint` + implémentations `ConstraintValidator`), la transformation des annotations de sécurité `@RolesAllowed` en `@PreAuthorize`, et l'ajout de commentaires de traçabilité `[REST-IN]`, `[REST-OUT]` et `[EJB-CALL]` dans le code généré.

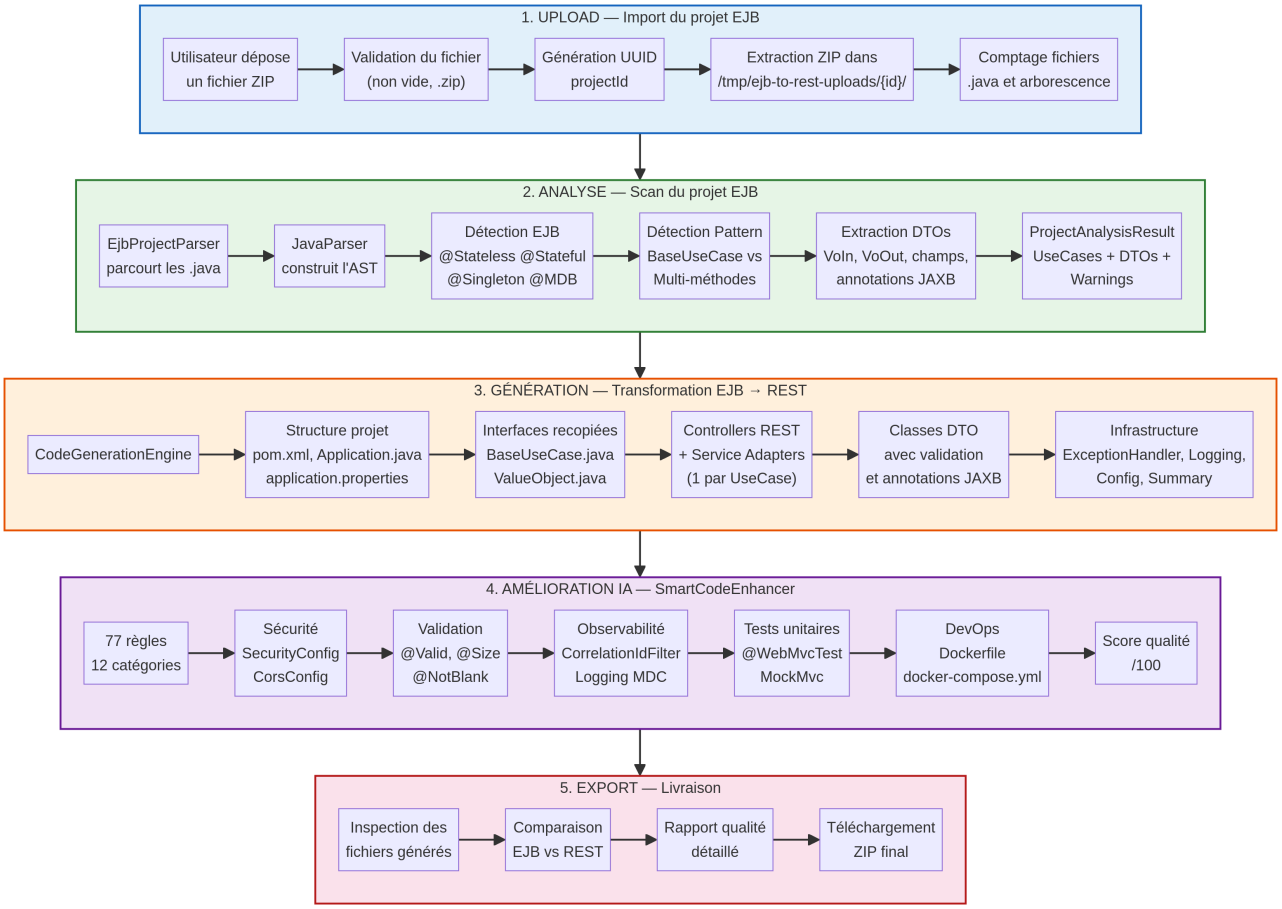
L'outil repose sur trois composants techniques majeurs. Le premier est le **EjbProjectParser**, qui utilise la bibliothèque JavaParser pour construire un arbre syntaxique abstrait (AST) de chaque fichier Java et en extraire les métadonnées structurelles

(annotations EJB, DTOs, enums, exceptions, validateurs, interfaces `@Remote`, champs, noms JNDI, annotations `@RolesAllowed` et `@XmlTransient`). Le deuxième est le **CodeGenerationEngine**, qui transforme ces métadonnées en code Java Spring Boot fonctionnel en appliquant 14 règles de gestion (G1 à G14), en générant les composants MDB event-driven, et en recopiant les types métier avec migration `javax` → `jakarta`. Le troisième est le **SmartCodeEnhancer**, un moteur de règles internes qui applique 77 règles réparties en 12 catégories pour enrichir le code généré avec les bonnes pratiques REST (sécurité, validation, observabilité, tests, DevOps), et qui transforme les annotations de sécurité EJB en annotations Spring Security.

---

2. Vue d'ensemble du pipeline

Le pipeline de transformation se décompose en cinq étapes principales, chacune correspondant à un écran de l'interface web et à un ensemble d'opérations techniques bien définies. Le diagramme ci-dessous illustre l'enchaînement global de ces étapes.



Le tableau suivant résume les caractéristiques de chaque étape du pipeline.

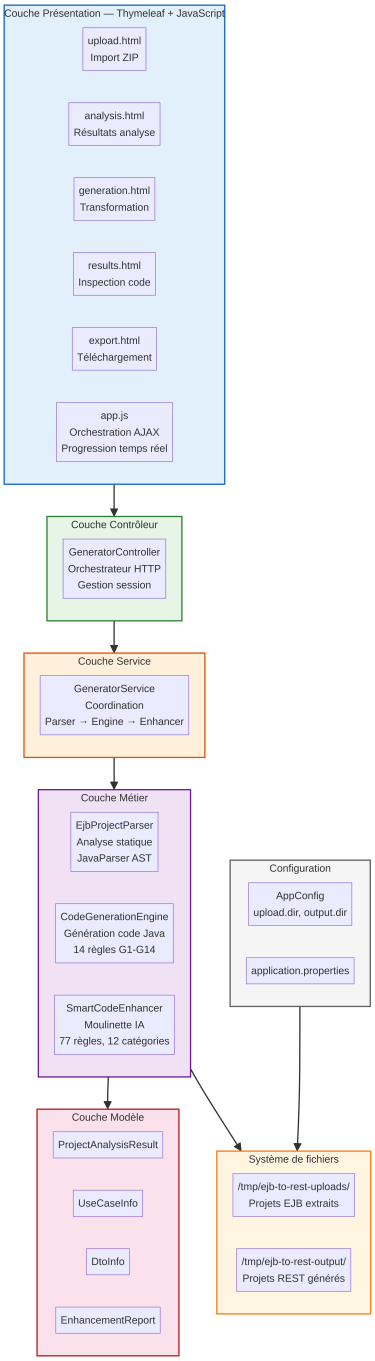
Étape	Nom	Endpoint HTTP	Composant principal	Entrée	Sortie
1	Upload	`POST /upload`	GeneratorController	Fichier ZIP du projet EJB	Arborescence extraite, projectId
2	Analyse	`POST /scan`	EjbProjectParser	Fichiers .java extraits	ProjectAnalysisResult (UseCases, DTOs)
3	Génération	`POST /api/generate`	CodeGenerationEngine	ProjectAnalysisResult	Projet Spring Boot complet
4	Amélioration	(inclus dans étape 3)	SmartCodeEnhancer	Projet généré brut	Projet enrichi + EnhancementReport
5	Export	`GET /download`	GeneratorController	Projet enrichi	Fichier ZIP téléchargeable

L'ensemble du processus est orchestré par le **GeneratorController**, qui maintient l'état de la session utilisateur (projectId, résultat d'analyse, fichiers générés, rapport d'amélioration) via la session HTTP. Le **GeneratorService** coordonne les appels entre le parseur, le moteur de génération et le moteur d'amélioration.

---

3. Architecture technique

L'application est structurée en couches selon les principes de l'architecture hexagonale. Le diagramme ci-dessous présente l'organisation des composants.



3.1 Couche Présentation

La couche présentation repose sur **Thymeleaf** pour le rendu côté serveur et **JavaScript** (fichier `app.js`) pour les interactions dynamiques. L'interface se compose de six vues principales : le tableau de bord (`dashboard.html`), l'import (`upload.html`), l'analyse (`analysis.html`), la génération (`generation.html`), les résultats (`results.html`) et l'export (`export.html`).

Le fichier `app.js` orchestre notamment la phase de transformation via la fonction `launchTransformation()`, qui lance en parallèle une animation de progression en six sous-étapes visuelles (parsing JavaParser, génération controllers, génération adapters/DTOs, SmartCodeEnhancer, tests unitaires, packaging) et un appel AJAX `POST /api/generate`. L'animation et la réponse serveur sont synchronisées avant de finaliser l'affichage.

3.2 Couche Contrôleur

Le **GeneratorController** est le point d'entrée unique pour toutes les requêtes HTTP. Il gère la session utilisateur et orchestre les redirections entre les différentes étapes du pipeline. Il expose les endpoints suivants :

Endpoint	Méthode	Rôle
`/upload`	POST	Réception et extraction du fichier ZIP
`/scan`	POST	Lancement de l'analyse statique
`/api/generate`	POST	Lancement de la génération + amélioration
`/api/file-content`	GET	Prévisualisation d'un fichier généré
`/api/diff`	GET	Comparaison EJB original vs REST généré
`/download`	GET	Téléchargement du projet ZIP

3.3 Couche Service

Le **GeneratorService** joue le rôle de coordinateur entre les trois composants métier. Lors de la génération, il exécute séquentiellement : (1) la création du répertoire de sortie, (2) l'appel au `CodeGenerationEngine.generateProject()`, puis (3) l'appel au `SmartCodeEnhancer.enhance()`. Il collecte les fichiers générés et le rapport d'amélioration pour les transmettre au contrôleur.

3.4 Couche Métier

Cette couche contient les trois composants fondamentaux de l'outil, détaillés dans les sections suivantes de ce document.

3.5 Configuration et système de fichiers

La classe **AppConfig** injecte les chemins de travail depuis `application.properties` :

Propriété	Valeur par défaut	Rôle
`app.upload.dir`	`/tmp/ejb-to-rest-uploads`	Stockage des projets EJB extraits
`app.output.dir`	`/tmp/ejb-to-rest-output`	Stockage des projets REST générés
`server.port`	`8080`	Port de l'application outil
`spring.servlet.multipart.max-file-size`	`100MB`	Taille maximale du fichier ZIP

---

4. Étape 1 — Upload du projet EJB

4.1 Processus fonctionnel

L'utilisateur accède à la page d'import et dépose un fichier ZIP contenant le projet EJB legacy. L'interface propose un mécanisme de drag-and-drop ainsi qu'un sélecteur de fichier classique. Le fichier doit être au format `.zip` et ne pas être vide.

4.2 Traitement technique

Lorsque le formulaire est soumis (`POST /upload`), le **GeneratorController** exécute les opérations suivantes dans l'ordre :

Le contrôleur vérifie d'abord que le fichier n'est pas vide. En cas de fichier vide, un message d'erreur est affiché et l'utilisateur est redirigé vers la page d'import. Ensuite, un identifiant unique de projet (**projectId**) est généré sous forme d'UUID. Le fichier ZIP est

ensuite extrait dans le répertoire `{app.upload.dir}/{projectId}/`. Le contrôleur effectue un comptage des fichiers totaux et des fichiers Java (`.java`), puis construit l'arborescence du projet sous forme d'une structure hiérarchique (fileTree).

Toutes ces informations sont stockées dans la **session HTTP** : `projectId`, `projectPath`, `fileTree`, `totalFiles`, `javaFiles`. L'utilisateur est ensuite redirigé vers la page d'import avec un message de succès, où il peut visualiser l'arborescence du projet extrait et les statistiques de base.

4.3 Données de session après upload

Attribut de session	Type	Description
`projectId`	String (UUID)	Identifiant unique du projet
`projectPath`	String	Chemin absolu du projet extrait
`fileTree`	String	Arborescence HTML du projet
`totalFiles`	int	Nombre total de fichiers
`javaFiles`	int	Nombre de fichiers .java

---

5. Étape 2 — Analyse statique du code EJB

5.1 Processus fonctionnel

L'utilisateur lance l'analyse en cliquant sur le bouton « Lancer l'analyse » depuis la page d'analyse. Le système parcourt tous les fichiers `.java` du projet extrait et en extrait les métadonnées structurelles.

5.2 Le composant EjbProjectParser

Le **EjbProjectParser** est le composant responsable de l'analyse statique. Il utilise la bibliothèque **JavaParser** pour construire l'arbre syntaxique abstrait (AST) de chaque fichier Java. Le parseur effectue deux passes sur le projet.

Première passe — Détection des EJB (UseCases) : Le parseur recherche les classes annotées `@Stateless`, `@Stateful`, `@Singleton` ou `@MessageDriven`. Pour les MDB, le parseur extrait désormais les méthodes publiques (notamment `onMessage`) et les propriétés d'activation (`destinationType`, `destinationLookup`). Pour chaque EJB non-MDB détecté, il détermine le **pattern d'utilisation** :

Pattern	Condition de détection	Génération
BaseUseCase	La classe implémente `BaseUseCase` et possède une méthode `execute(ValueObject)`	1 controller + 1 service adapter avec méthode `execute()`
Multi-méthodes	La classe possède plusieurs méthodes publiques métier	1 controller avec N routes (une par méthode)

Pour le pattern BaseUseCase, le parseur identifie les DTOs d'entrée (VoIn) et de sortie (VoOut) en analysant les conventions de nommage : `{NomUseCase}VoIn` et `{NomUseCase}VoOut`. Il résout également le nom JNDI en analysant les annotations `@EJB` ou en appliquant la convention `java:global/{nomClasse}`.

Pour le pattern multi-méthodes, le parseur extrait chaque méthode publique avec ses paramètres (nom, type) et son type de retour. Il identifie les DTOs utilisés en paramètres et en retour. Pour chaque méthode, le parseur extrait également les **annotations** `@RolesAllowed` (rôles de sécurité EJB) et les **clauses** `throws` (exceptions déclarées). Si la classe implémente une **interface** `@Remote`, le nom de cette interface est enregistré pour permettre le cast typé dans le ServiceAdapter généré.

Deuxième passe — Extraction des types métier : En plus des DTOs, le parseur extrait désormais les **enums métier** (classes `enum` avec ou sans `@XmlEnum`), les **exceptions custom** (classes étendant `Exception` ou `RuntimeException`), les **validateurs custom**

(annotations `@interface` portant `@Constraint` et classes implémentant `ConstraintValidator`), et les **interfaces `@Remote`**. Le code source complet de chaque type est capturé pour permettre la recopie avec migration `javax` → `jakarta`.

Extraction des DTOs : Le parseur recherche les classes dont le nom se termine par `VoIn`, `VoOut`, `Dto`, `DTO`, `Input`, `Output`, `Request` ou `Response`. Pour chaque DTO, il extrait :

Information extraite	Méthode de détection
Champs (nom, type, visibilité)	Analyse des déclarations de champs dans l'AST
Annotations JAXB	`@XmlRootElement`, `@XmlAccessorType`, `@XmlType`, `@XmlElement`, `@XmlAttribute`
Champs required	`@XmlElement(required = true)` ou `@NotNull`
`@XmlTransient`	Détection de `@XmlTransient` sur les champs (transformé en `@JsonIgnore`)
Format de sérialisation	Présence d'annotations JAXB → XML, sinon JSON
Classe parente	Clause `extends` dans la déclaration

Le parseur génère également des **warnings** pour les cas nécessitant une attention particulière : EJB `@Stateful` (état conversationnel non reproductible en REST), EJB sans DTO détecté, entités JPA détectées. Les MDB ne génèrent plus de warning depuis la version 2.0 car ils sont désormais entièrement transformés.

5.3 Résultat de l'analyse — ProjectAnalysisResult

Le résultat de l'analyse est encapsulé dans un objet `ProjectAnalysisResult` qui contient :

Propriété	Type	Description
`useCases`	`List<UseCaseInfo>`	Liste des EJB détectés avec leurs métadonnées
`dtos`	`List<DtoInfo>`	Liste des DTOs détectés avec leurs champs
`enums`	`List<EnumInfo>`	Enums métier détectés (nom, constantes, source)
`exceptions`	`List<ExceptionInfo>`	Exceptions custom détectées (nom, classe parente, source)
`validators`	`List<ValidatorInfo>`	Validateurs custom détectés (annotation + implémentation)
`remoteInterfaces`	`Map<String, String>`	Interfaces @Remote (nom → code source)
`totalFilesAnalyzed`	int	Nombre de fichiers analysés
`jpaEntityCount`	int	Nombre d'entités JPA détectées
`warnings`	`List<String>`	Avertissements générés
`sourceBasePackage`	String	Package racine détecté

Chaque **UseCaseInfo** contient : le nom de la classe, le type EJB (`STATELESS`, `STATEFUL`, `SINGLETON`, `MESSAGE_DRIVEN`), le pattern (`BASE_USE_CASE`, `MULTI_METHOD`), le nom JNDI, l'endpoint REST dérivé, les noms des DTOs d'entrée/sortie, le format de sérialisation, le Javadoc, la liste des méthodes publiques (pour le pattern multi-méthodes), les **rôles `@RolesAllowed`** au niveau classe, et le **nom de l'interface `@Remote`** implémentée. Chaque **MethodInfo** contient également les rôles `@RolesAllowed` au niveau méthode et les exceptions déclarées dans la clause `throws`.

Chaque **DtoInfo** contient : le nom de la classe, le package, la liste des champs (`FieldInfo` avec nom, type, annotations JAXB, required, indicateur `@XmlTransient`), les indicateurs d'annotations JAXB au niveau classe, et la classe parente.

---

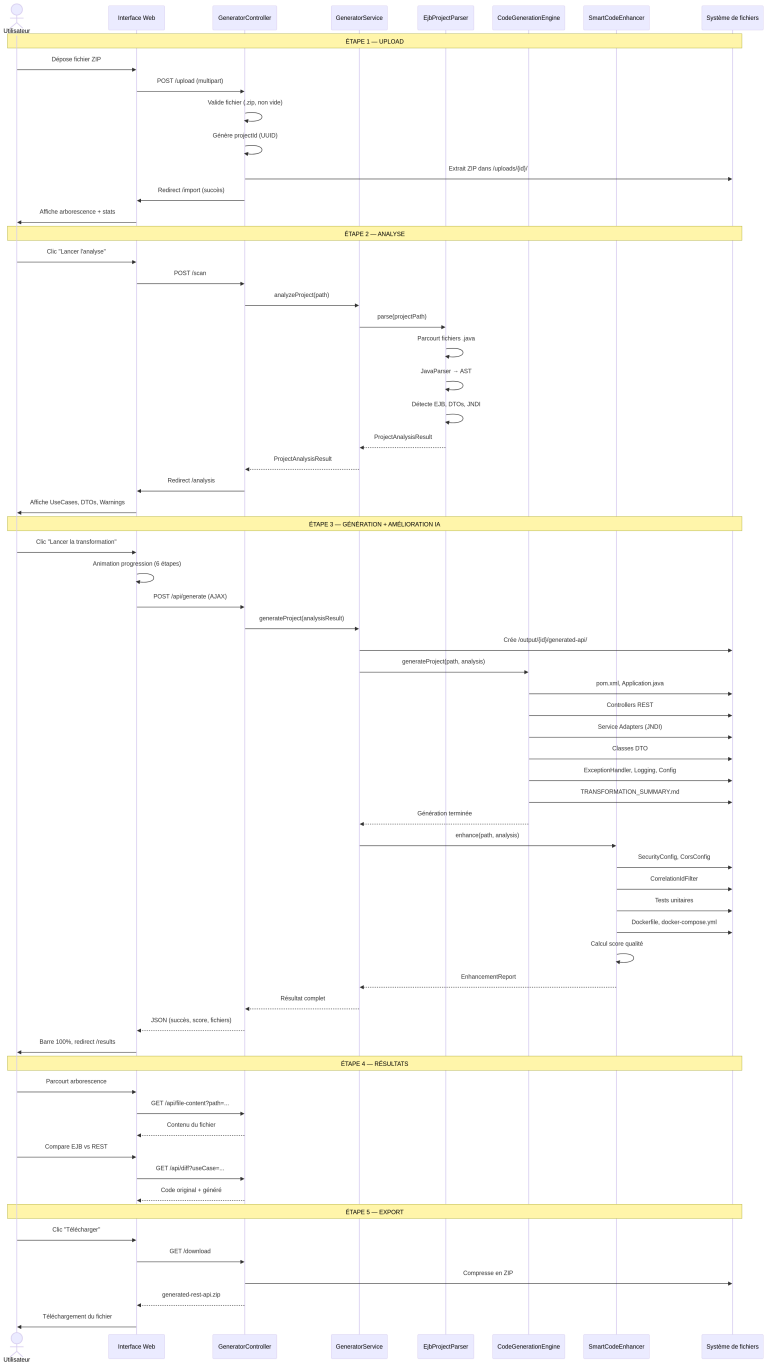
6. Étape 3 — Génération du projet REST

6.1 Processus fonctionnel

L'utilisateur lance la transformation depuis la page de génération. L'interface affiche une barre de progression animée avec six sous-étapes visuelles pendant que le serveur exécute la génération et l'amélioration en arrière-plan.

6.2 Diagramme de séquence

Le diagramme ci-dessous illustre l'enchaînement complet des appels entre les composants lors de la génération.



6.3 Le composant CodeGenerationEngine

Le **CodeGenerationEngine** est le moteur de génération de code. Il prend en entrée le `ProjectAnalysisResult` et produit un projet Spring Boot complet dans le répertoire de sortie. La génération s'effectue en plusieurs phases séquentielles.

Phase 1 — Structure du projet : Le moteur génère le `pom.xml` avec les dépendances Spring Boot 3.2.5 (Web, Validation, AOP, et conditionnellement JAXB/Jackson XML si des annotations JAXB ont été détectées), la classe `Application.java` avec `@SpringBootApplication`, et le fichier `application.properties` avec la configuration JNDI, logging et Swagger.

Phase 2 — Interfaces recopiées : Le moteur génère les interfaces `BaseUseCase` et `ValueObject` dans le package `ejb.interfaces`, qui sont des copies des interfaces du projet EJB source. Ces interfaces permettent le cast type lors du lookup JNDI.

Phase 2b — Types métier recopiés (nouveau en v2.0) : Le moteur génère les enums métier, les exceptions custom et les validateurs custom dans des packages dédiés (`enums/`, `exception/`, `validation/`). Pour les **enums**, le code source est recopié avec migration `javax.xml.bind` → `jakarta.xml.bind`, et les annotations `@JsonValue` / `@JsonCreator` sont ajoutées pour la compatibilité Jackson. Pour les **exceptions**, le code source est recopié tel quel avec migration des imports. Pour les **validateurs**, l'annotation `@interface` et la classe `ConstraintValidator` sont recopiées avec migration `javax.validation` → `jakarta.validation`, en préservant la logique de validation originale.

Phase 3 — Controllers REST : Pour chaque UseCase, le moteur génère un controller REST. Le choix entre les deux modèles de controller dépend du pattern détecté :

Pour le **pattern BaseUseCase**, le controller expose un unique endpoint POST avec la méthode `execute()`. Le corps de la méthode appelle le service adapter, qui effectue le lookup JNDI. Le code HTTP de retour est déterminé par la règle G6 (voir section 9). Des commentaires de traçabilité `// [REST-IN]` et `// [REST-OUT]` sont ajoutés dans les logs du controller.

Pour le **pattern multi-méthodes**, le controller expose une route par méthode publique de l'EJB. Chaque route est annotée avec la méthode HTTP appropriée (`@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`) selon la sémantique du nom de la méthode (règle G6). Les paramètres sont annotés avec `@PathVariable`, `@RequestParam` ou `@RequestBody` selon leur type et leur nom (règle G8).

Phase 4 — Service Adapters : Pour chaque UseCase, le moteur génère un service adapter annoté `@Service`. Ce composant encapsule le lookup JNDI vers l'EJB distant. Il utilise les propriétés externalisées (`ejb.jndi.provider.url`, `ejb.jndi.factory`) pour construire le contexte JNDI, effectue le lookup, cast le résultat en `BaseUseCase`, et appelle la méthode `execute()`. Le contexte JNDI est fermé dans un bloc `finally` pour éviter les fuites de ressources. Un commentaire `// [EJB-CALL]` est ajouté dans le log d'appel JNDI. Pour les services multi-méthodes implémentant une **interface `@Remote`**, le cast est effectué vers cette interface typée (ex: `(AccountServiceRemote) lookupEjb()`) au lieu d'un cast générique.

Phase 5 — Classes DTO : Pour chaque DTO détecté, le moteur génère une classe Java avec les caractéristiques suivantes : implémentation de `ValueObject` (pour VoIn/VoOut) ou `Serializable` (pour les autres DTOs), `serialVersionUID = 1L`, annotations JAXB préservées et migrées de `javax` vers `jakarta` (règle G3), annotations de validation (`@NotBlank`, `@NotNull`, `@Size`) sur les champs required (règle G13), getters/setters explicites, constructeur par défaut, et méthode `toString()`.

Phase 5b — MDB event-driven (nouveau en v2.0) : Pour chaque `@MessageDriven` Bean, le moteur génère 4 composants Spring : un **Controller REST async** (endpoint POST, réponse `202 Accepted` avec `correlationId`), un **Event Spring** (étendant `ApplicationEvent`), un **EventListener async** (`@Async @EventListener` contenant la logique métier), et un **ServiceAdapter** (pont JNDI). Le nommage est intelligent : le suffixe "MDB" est retiré et la redondance est évitée (ex: `AuditEventMDB` → `AuditEvent`, `AuditEventListener`, `AuditEventController`).

Phase 6 — Infrastructure : Le moteur génère les composants transversaux : `GlobalExceptionHandler` avec `@ControllerAdvice` (règle G9) enrichi avec les handlers pour les exceptions custom détectées dans le projet source, `LoggingAspect` avec `@Aspect` pour le logging AOP, `ContentNegotiationConfig` si des annotations JAXB ont été détectées (règle G12), `EjbLookupConfig` pour la configuration JNDI centralisée.

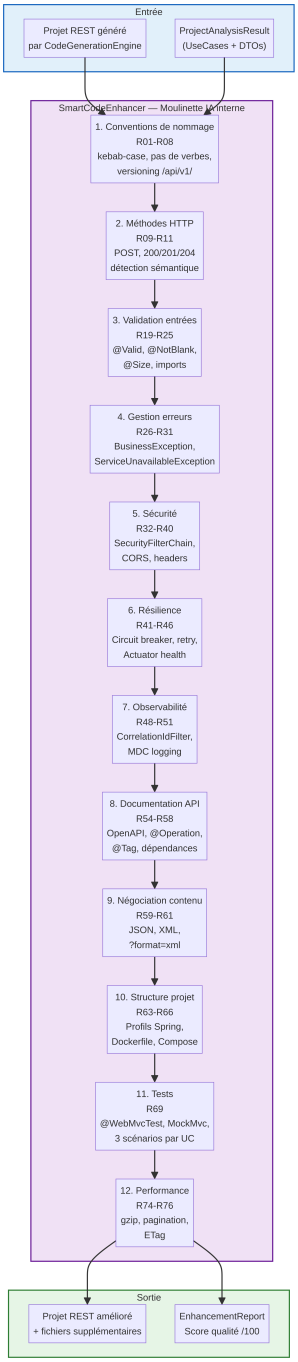
Phase 7 — Documentation : Le moteur génère le fichier `TRANSFORMATION_SUMMARY.md` (règle G14) avec le tableau de mapping détaillé (EJB source → endpoint REST, méthode HTTP, code HTTP), et le fichier `README.md` avec les instructions de compilation, d'exécution et d'accès à Swagger UI.

---

7. Étape 4 — Amélioration par la moulinette IA (SmartCodeEnhancer)

7.1 Présentation

Le **SmartCodeEnhancer** est un moteur de règles internes qui fonctionne entièrement en local, sans appel à une IA externe. Il analyse et améliore automatiquement le code généré par le `CodeGenerationEngine` en appliquant **77 règles** réparties en **12 catégories**, basées sur les bonnes pratiques REST API (OWASP, Postman, Spring Boot, OpenAPI). Ce moteur garantit l'absence d'hallucinations et la reproductibilité des résultats.



7.2 Fonctionnement

Le SmartCodeEnhancer reçoit en entrée le chemin du projet généré et le `ProjectAnalysisResult`. Il parcourt les fichiers générés et applique ses règles séquentiellement, catégorie par catégorie. Chaque règle peut :

- **Vérifier** une condition (audit) et la rapporter dans le rapport

- **Modifier** un fichier existant (ajout d'annotations, d'imports, de code)
- **Créer** un nouveau fichier (SecurityConfig, tests, Dockerfile…)

Chaque application de règle est enregistrée dans un **EnhancementReport** avec : l'identifiant de la règle (R01 à R76), la catégorie, la sévérité (Info, Suggestion, Warning, Critical), une description, le fichier concerné, et un indicateur d'application.

7.3 Les 12 catégories de règles

Le tableau ci-dessous présente une vue synthétique des 12 catégories avec leurs règles principales et les fichiers impactés.

Cat.	Nom	Règles	Sévérité max	Fichiers impactés
1	Conventions de nommage	R01, R03, R08	Info	Controllers
2	Méthodes HTTP et codes de statut	R09, R11	Warning	Controllers
3	Validation des entrées	R19-R25	Critical	Controllers, DTOs
4	Gestion des erreurs	R26-R31	Warning	ExceptionHandler, Exceptions
5	Sécurité	R32-R40	Critical	SecurityConfig, CorsConfig
6	Résilience	R41-R46	Warning	application.properties
7	Observabilité	R48-R51	Critical	CorrelationIdFilter, properties
8	Documentation API	R54-R58	Critical	pom.xml, Controllers
9	Négociation de contenu	R59-R61	Info	ContentNegotiationConfig
10	Structure projet	R63-R66	Suggestion	Profils, Dockerfile, Compose
11	Tests	R69	Warning	Tests unitaires
12	Performance	R74-R76	Suggestion	application.properties

7.4 Détail des catégories

Catégorie 1 — Conventions de nommage : Cette catégorie vérifie que les endpoints REST utilisent le format kebab-case (R01), ne contiennent pas de verbes dans les URLs (R03), et incluent un versioning `/api/v1/` (R08). Ces règles sont principalement des vérifications d'audit, car le `CodeGenerationEngine` applique déjà ces conventions lors de la génération.

Catégorie 2 — Méthodes HTTP et codes de statut : La règle R11 est l'une des plus importantes. Elle détermine le code HTTP approprié en analysant le nom de la classe UseCase. Les opérations de **création** (détectées par les mots-clés `create`, `creer`, `souscrire`, `ajouter`, `inscrire`, `enregistrer`, `add`, `insert`, `register`, `open`, `save`, `nouveau` dans le nom de classe) sont mises en `201 Created`. Les opérations de **suppression** (détectées par `delete`, `supprimer`, `remove`, `close`, `cancel`, `annuler`) sont mises en `204 No Content`. Toutes les autres opérations (consultation, transfert, exécution) restent en `200 OK`. La détection se fait uniquement sur le nom de classe et non sur le Javadoc, car ce dernier peut contenir des faux positifs (par exemple, un UseCase de virement dont le Javadoc mentionne « la création des écritures comptables »).

Catégorie 3 — Validation des entrées : La règle R19 ajoute `@Valid` sur tous les paramètres `@RequestBody` des controllers. Les règles R20-R25 ajoutent des annotations de validation sur les champs String des DTOs d'entrée : `@NotBlank` pour les champs obligatoires, `@Size(max = 255)` pour tous les champs String. Les imports sont ajoutés individuellement (pas en bloc) pour éviter les doublons, et une vérification d'idempotence empêche la duplication des annotations lors de multiples exécutions.

Catégorie 4 — Gestion des erreurs : Cette catégorie vérifie la présence du `GlobalExceptionHandler` (R26), ajoute un handler pour `MethodArgumentNotValidException` qui retourne les erreurs de validation en une seule réponse (R27, R31), crée une `BusinessException` pour les erreurs métier avec code HTTP 409 Conflict (R29), et crée une `ServiceUnavailableException` pour les erreurs JNDI avec code HTTP 503 (R30).

Catégorie 5 — Sécurité : La règle R32 crée un `SecurityFilterChain` avec mode stateless (pas de session HTTP), CSRF désactivé (API REST stateless), endpoints publics (`/api/**`, `/actuator/health`, `/swagger-ui/**`), et headers de sécurité (`X-Content-`

`Type-Options: nosniff`, `X-Frame-Options: DENY`). La règle R33 crée un `CorsConfig` avec des origines explicites (jamais de wildcard `*`).

Catégorie 6 — Résilience : Cette catégorie ajoute la configuration Spring Boot Actuator (R46) avec les endpoints `health`, `info` et `metrics`, et recommande l'ajout de circuit breaker (R41), retry avec backoff exponentiel (R42) et timeout (R43) sur les appels JNDI.

Catégorie 7 — Observabilité : La règle R48 crée un `CorrelationIdFilter` qui génère un identifiant de corrélation (`X-Request-ID`) pour chaque requête, le propage via le MDC (Mapped Diagnostic Context) de SLF4J, et le retourne dans les en-têtes de réponse. La règle R50 modifie le pattern de logging pour inclure cet identifiant.

Catégorie 8 — Documentation API : Cette catégorie ajoute les annotations OpenAPI (`@Operation`, `@ApiResponses`, `@Tag`) sur les controllers (R55), et les dépendances Maven nécessaires : `springdoc-openapi-starter-webmvc-ui` (R54), `spring-boot-starter-security` (R32b), `spring-boot-starter-actuator` (R46b), `spring-boot-starter-validation` (R19b).

Catégorie 9 — Négociation de contenu : Cette catégorie vérifie que le JSON est supporté par défaut (R59), que le XML est supporté si des annotations JAXB ont été détectées (R60), et que la négociation par paramètre (`?format=xml`) est activée (R61).

Catégorie 10 — Structure projet : Cette catégorie crée les profils Spring `application-dev.properties` et `application-prod.properties` (R64), un `Dockerfile` multi-stage avec build Maven et runtime JRE Alpine en utilisateur non-root (R65), et un `docker-compose.yml` avec health check (R66).

Catégorie 11 — Tests : La règle R69 génère un test unitaire par controller avec `@WebMvcTest` et `MockMvc`. Chaque test contient trois scénarios : exécution réussie (vérifie le code HTTP attendu : 200 ou 201 selon la sémantique), requête invalide (vérifie le code 4xx), et erreur service (vérifie le code 500). Le service adapter est mocké avec `@MockBean`.

Catégorie 12 — Performance : Cette catégorie active la compression gzip pour les réponses supérieures à 1 Ko (R75), et recommande la pagination pour les endpoints de type liste (R74) et le caching HTTP via ETag (R76).

---

8. Étape 5 — Résultats et export

8.1 Page de résultats

Après la génération, l'utilisateur est redirigé vers la page de résultats qui affiche quatre indicateurs de synthèse : le nombre de fichiers générés, le nombre de controllers REST, le nombre de tests unitaires, et le score de qualité SmartCodeEnhancer.

La page propose une **arborescence cliquable** de tous les fichiers générés. En cliquant sur un fichier, son contenu est affiché dans un panneau de prévisualisation via l'endpoint `GET /api/file-content?path=...`.

Un **comparateur côte-à-côte** permet de visualiser le code EJB original à gauche et le code REST généré à droite pour chaque UseCase, via l'endpoint `GET /api/diff?useCase=...`.

8.2 Export

La page d'export propose un bouton de téléchargement qui déclenche `GET /download`. Le contrôleur compresse le répertoire du projet généré en un fichier ZIP (`generated-rest-api.zip`) et l'envoie en téléchargement. La page affiche également un guide de démarrage rapide en quatre étapes : décompression, compilation Maven, exécution du JAR, et accès à Swagger UI.

---

9. Règles de gestion (G1-G14)

Les règles de gestion G1 à G14 sont appliquées par le **CodeGenerationEngine** lors de la phase de génération du code. Elles définissent les conventions de transformation entre le monde EJB et le monde REST.

Règle	Nom	Description détaillée
G1	Imports individuels	Utiliser des imports Java individuels (pas de wildcard `*`) pour éviter les conflits de noms entre packages. Les imports sont collectés dans un `TreeSet` pour garantir l'ordre alphabétique et la déduplication.
G2	Déduplication des annotations	Utiliser des `LinkedHashSet` pour les annotations de champ afin de garantir l'unicité tout en préservant l'ordre d'insertion. Empêche les doublons `@NotBlank`, `@Size`, `@XmlElement`.
G3	Migration javax → jakarta	Convertir toutes les références `javax.xml.bind` en `jakarta.xml.bind` pour la compatibilité Jakarta EE 10+. Ajouter `@JacksonXmlRootElement` en complément de `@XmlRootElement` pour le support Jackson XML.
G4	Commentaire EJB type	Ajouter un commentaire d'avertissement dans le code généré si l'EJB source est `@Stateful` (état conversationnel non reproductible), `@Singleton` (scope singleton préservé), ou `@MessageDriven` (non exposable via REST).
G5	Multi-méthodes	Si l'EJB possède plusieurs méthodes publiques métier (pas le pattern BaseUseCase), générer une route REST par méthode avec un sous-chemin dérivé du nom de la méthode en kebab-case.
G6	HTTP status codes	Mapper le code HTTP selon la sémantique de l'opération. Les méthodes GET retournent 200, les POST de création retournent 201, les DELETE retournent 204, et les autres POST retournent 200. La détection se fait par analyse du nom de la méthode.
G7	Type de retour byte[]	Si une méthode retourne `byte[]`, ajouter les en-têtes `Content-Disposition: attachment` et `Content-Type: application/octet-stream` dans la réponse.
G8	Annotations de paramètres	Résoudre automatiquement l'annotation de chaque paramètre : `@RequestBody` pour les DTOs et objets complexes, `@PathVariable` pour les paramètres dont le nom contient « id », `@RequestParam` pour les types simples en GET/DELETE.
G9	Gestion des erreurs	Générer un `GlobalExceptionHandler` avec `@ControllerAdvice` qui mappe les exceptions vers les codes HTTP appropriés : `NamingException` → 503, « not found » → 404, « already exists » → 409, « unauthorized » → 401, « insufficient » → 422.
G10	JNDI lookup	Générer un Service Adapter par UseCase qui encapsule le lookup JNDI avec `InitialContext`, `Properties`, cast vers `BaseUseCase`, et fermeture du contexte dans un bloc `finally`. Les propriétés JNDI sont externalisées via `@Value`.
G11	Swagger/OpenAPI	Ajouter les annotations `@Operation(summary = ...)`, `@ApiResponse(responseCode = ...)`, et `@ApiResponses` sur chaque endpoint REST pour la documentation automatique Swagger.
G12	Négociation de contenu	Si des annotations JAXB ont été détectées dans le projet source, ajouter `produces = {"application/json", "application/xml"}` et `consumes = {"application/json", "application/xml"}` sur les endpoints, et générer un `ContentNegotiationConfig`.
G13	Validation des entrées	Ajouter `@Valid` sur les paramètres `@RequestBody` des controllers, et `@NotBlank` / `@NotNull` / `@Size` sur les champs required des DTOs d'entrée.
G14	Résumé de transformation	Générer un fichier `TRANSFORMATION_SUMMARY.md` avec le tableau de mapping complet (EJB → REST) et un `README.md` avec les instructions de démarrage.

---

10. Catalogue complet des 77 règles SmartCodeEnhancer

Le tableau ci-dessous présente l'ensemble des règles appliquées par le SmartCodeEnhancer, organisées par catégorie.

ID	Catégorie	Sévérité	Description	Fichier impacté
R01	Nommage	Info	Endpoints en kebab-case	Controllers
R03	Nommage	Info	Pas de verbes dans les URLs	Controllers
R08	Nommage	Suggestion	Versioning /api/v1/	Controllers
R09	HTTP	Info	execute() mappé sur POST	Controllers
R11	HTTP	Warning	Code HTTP selon sémantique (200/201/204)	Controllers
R19	Validation	Critical	@Valid sur @RequestBody	Controllers
R20	Validation	Warning	@NotBlank + @Size sur champs String	DTOs
R26	Erreurs	Info	GlobalExceptionHandler présent	ExceptionHandler
R27	Erreurs	Warning	Handler MethodArgumentNotValidException	ExceptionHandler
R29	Erreurs	Warning	BusinessException (409 Conflict)	BusinessException.java
R30	Erreurs	Warning	ServiceUnavailableException (503)	ServiceUnavailableException.java
R31	Erreurs	Info	Erreurs de validation en une réponse	ExceptionHandler
R32	Sécurité	Critical	SecurityFilterChain (stateless, CSRF off)	SecurityConfig.java
R33	Sécurité	Critical	CorsConfig origines explicites	CorsConfig.java
R34	Sécurité	Info	X-Content-Type-Options: nosniff	SecurityConfig.java
R35	Sécurité	Info	X-Frame-Options: DENY	SecurityConfig.java
R36	Sécurité	Suggestion	Cache-Control: no-store	application.properties
R40	Sécurité	Info	CSRF désactivé (API stateless)	SecurityConfig.java
R41	Résilience	Suggestion	Circuit breaker recommandé	ServiceAdapters
R42	Résilience	Suggestion	Retry avec backoff exponentiel	ServiceAdapters
R43	Résilience	Suggestion	Timeout sur appels JNDI	ServiceAdapters
R46	Résilience	Warning	Actuator health/info/metrics	application.properties
R48	Observabilité	Critical	CorrelationIdFilter (X-Request-ID)	CorrelationIdFilter.java
R49	Observabilité	Info	LoggingAspect présent	LoggingAspect.java
R50	Observabilité	Warning	Correlation ID dans le logging	application.properties
R54	Documentation	Critical	Dépendance springdoc-openapi	pom.xml
R55	Documentation	Warning	@Operation, @ApiResponses, @Tag	Controllers
R59	Contenu	Info	JSON supporté par défaut	ContentNegotiationConfig
R60	Contenu	Info	XML supporté (si JAXB)	ContentNegotiationConfig
R61	Contenu	Info	Négociation par paramètre	ContentNegotiationConfig
R63	Structure	Info	Configuration externalisée	application.properties
R64	Structure	Suggestion	Profils Spring (dev, prod)	application-*.properties
R65	Structure	Suggestion	Dockerfile multi-stage	Dockerfile
R66	Structure	Suggestion	docker-compose.yml	docker-compose.yml
R69	Tests	Warning	Test unitaire par controller	*ControllerTest.java

ID	Catégorie	Sévérité	Description	Fichier impacté
R74	Performance	Suggestion	Pagination recommandée	Controllers
R75	Performance	Suggestion	Compression gzip	application.properties
R76	Performance	Suggestion	ETag/If-None-Match	Controllers
R19b	Validation	Critical	Dépendance starter-validation	pom.xml
R32b	Sécurité	Critical	Dépendance starter-security	pom.xml
R46b	Résilience	Critical	Dépendance starter-actuator	pom.xml
R64b	Structure	Suggestion	Profil application-prod.properties	application-prod.properties

---

11. Transformation des Message-Driven Beans (MDB)

Depuis la version 2.0, les MDB ne sont plus ignorés mais entièrement transformés en composants Spring event-driven. Cette transformation préserve la sémantique asynchrone des MDB tout en les rendant accessibles via HTTP.

11.1 Architecture de transformation

Chaque `@MessageDriven` Bean est transformé en **4 composants Spring** :

Composant généré	Rôle	Annotation clé
Controller REST async	Endpoint POST qui reçoit les messages via HTTP	`@RestController`, `@PostMapping`
Event Spring	Objet événement typé pour découpler réception et traitement	`extends ApplicationEvent`
EventListener async	Traitement asynchrone de l'événement	`@Async`, `@EventListener`
ServiceAdapter	Pont JNDI vers le MDB legacy	`@Service`

11.2 Flux de traitement

Le flux de traitement d'un message MDB transformé suit les étapes suivantes :

1. Le client envoie un POST HTTP au controller (ex: `POST /api/events/audit-event`)
2. Le controller publie un `ApplicationEvent` via `ApplicationEventPublisher`
3. Le client reçoit immédiatement un `202 Accepted` avec un `correlationId` unique
4. Le listener asynchrone (`@Async @EventListener`) traite l'événement en arrière-plan
5. Le listener appelle le ServiceAdapter qui effectue le lookup JNDI vers le MDB legacy

11.3 Nommage intelligent

Le suffixe "MDB" est automatiquement retiré des noms de classes générées, et la redondance est évitée :

MDB Source	Controller	Event	Listener
`AuditEventMDB`	`AuditEventController`	`AuditEvent`	`AuditEventListener`
`TransactionNotificationMDB`	`TransactionNotificationController`	`TransactionNotificationEvent`	`TransactionNotificationEve`

Le SmartCodeEnhancer protège les controllers MDB : la règle R11 (cohérence HTTP) ne modifie pas le code `202 Accepted` des controllers MDB.

---

12. Extraction et génération des types métier (Axe 1)

L'axe 1 du cahier d'évolution v2.0 concerne l'extraction et la recopie des types métier qui étaient précédemment ignorés par l'outil.

12.1 Enums métier

Le parseur détecte les classes `enum` du projet source (avec ou sans `@XmlEnum`) et les recopié dans le package `enums/` du projet généré. La migration `javax.xml.bind` → `jakarta.xml.bind` est appliquée automatiquement. Les annotations `@JsonValue` et `@JsonCreator` sont ajoutées pour la compatibilité Jackson.

Enum source	Enum généré	Transformations
`AccountStatus` (avec `@XmlEnum`)	`AccountStatus`	Migration JAXB, ajout `@JsonValue` / `@JsonCreator`
`TransactionType` (sans JAXB)	`TransactionType`	Recopie directe

12.2 Exceptions custom

Le parseur détecte les classes étendant `Exception` ou `RuntimeException` et les recopié dans le package `exception/` du projet généré. Le `GlobalExceptionHandler` est automatiquement enrichi avec un `@ExceptionHandler` pour chaque exception custom détectée, mappé vers le code HTTP le plus pertinent :

Pattern dans le nom	Code HTTP	Exemple
"NotFound", "EntityNotFound"	404 Not Found	`EntityNotFoundException`
"Duplicate", "AlreadyExists"	409 Conflict	`DuplicateEntityException`
"Unauthorized", "Forbidden"	403 Forbidden	`UnauthorizedOperationException`
"Insufficient", "Balance"	422 Unprocessable Entity	`InsufficientBalanceException`
"Unavailable", "Timeout"	503 Service Unavailable	`ServiceUnavailableException`
"Closed", "Locked"	422 Unprocessable Entity	`AccountClosedException`
Autres	400 Bad Request	`BusinessException`

12.3 Validateurs custom

Le parseur détecte les annotations `@interface` portant `@Constraint` et les classes implémentant `ConstraintValidator`. Les deux fichiers (annotation + implémentation) sont recopiés dans le package `validation/` avec migration `javax.validation` → `jakarta.validation`. La logique de validation originale (méthode `isValid()`) est intégralement préservée.

Validateur source	Fichiers générés	Logique préservée
`@ValidIBAN` + `IBANValidator`	`ValidIBAN.java` + `IBANValidator.java`	Regex IBAN, longueur 15-34
`@ValidRIB` + `RIBValidator`	`ValidRIB.java` + `RIBValidator.java`	Format RIB, clé de contrôle

12.4 Transformation @XmlTransient → @JsonIgnore

Les champs annotés `@XmlTransient` dans les DTOs source sont transformés en `@JsonIgnore` dans les DTOs générés. L'import `com.fasterxml.jackson.annotation.JsonIgnore` est ajouté automatiquement. Cette transformation est effectuée par le SmartCodeEnhancer (règle R20b) après la génération initiale.

---

13. Commentaires de traçabilité (Axe 2)

L'axe 2 ajoute des commentaires de traçabilité dans le code généré pour faciliter le débogage et la compréhension du flux d'exécution entre les couches REST et EJB.

13.1 Marqueurs de traçabilité

Trois marqueurs sont utilisés :

Marqueur	Emplacement	Description	Exemple de log
`[REST-IN]`	Controller (entrée de méthode)	Log d'entrée dans l'endpoint REST	`log.info("[REST-IN] POST /api/create-account")`
`[REST-OUT]`	Controller (sortie de méthode)	Log de sortie avec résultat	`log.info("[REST-OUT] POST /api/create-account -> 201")`
`[EJB-CALL]`	ServiceAdapter (appel JNDI)	Log d'appel vers l'EJB distant	`log.info("[EJB-CALL] JNDI lookup: java:global/CreateAccountUC")`

13.2 Couverture

Les commentaires de traçabilité sont ajoutés dans :

- Tous les controllers REST (pattern BaseUseCase et multi-méthodes)
- Tous les controllers MDB event-driven
- Tous les service adapters (BaseUseCase et multi-méthodes)

---

14. Transformation des annotations de sécurité (Axe 3)

L'axe 3 transforme les annotations de sécurité EJB en annotations Spring Security.

14.1 @RolesAllowed → @PreAuthorize

Les annotations `@RolesAllowed` détectées sur les classes EJB sont transformées en `@PreAuthorize` Spring Security sur les controllers générés. La transformation suit les règles suivantes :

Annotation EJB	Annotation Spring Security générée
`@RolesAllowed("ADMIN")`	`@PreAuthorize("hasRole('ADMIN')")`
`@RolesAllowed({"ADMIN", "OPERATOR"})`	`@PreAuthorize("hasAnyRole('ADMIN','OPERATOR')")`

L'import `org.springframework.security.access.prepost.PreAuthorize` est ajouté automatiquement, et l'annotation `@EnableMethodSecurity` est vérifiée dans la configuration de sécurité.

14.2 Interface @Remote et cast typé

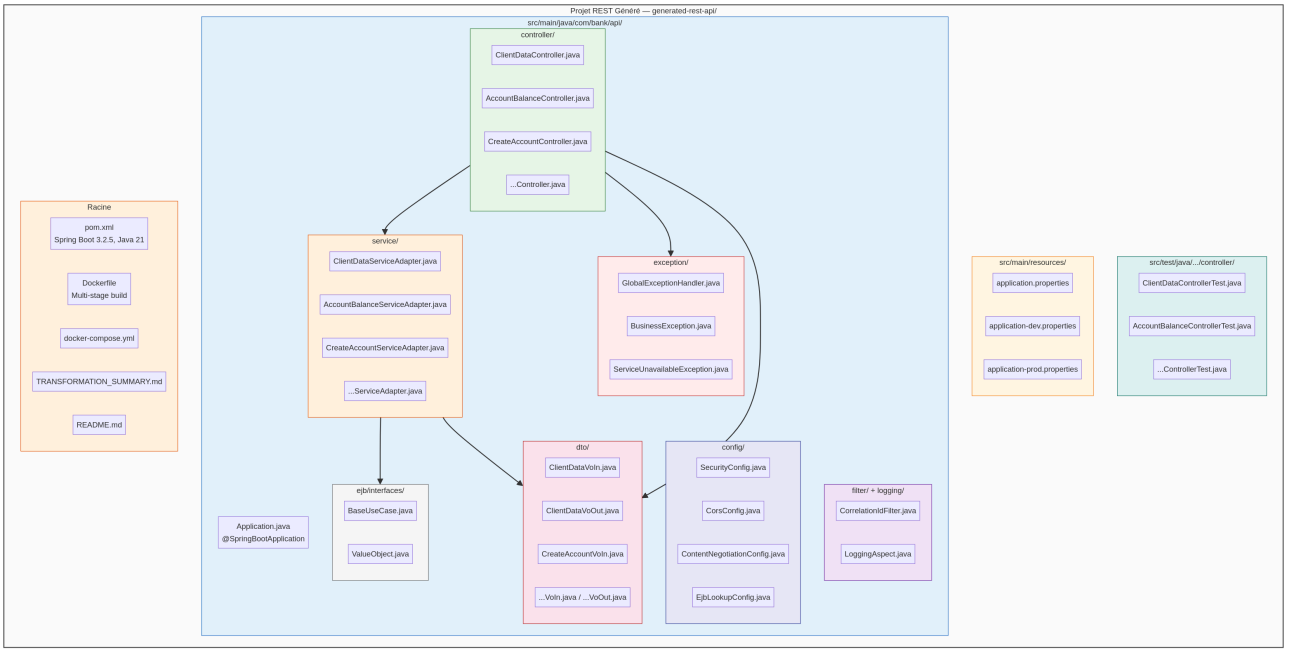
Pour les services multi-méthodes implémentant une interface `@Remote`, le ServiceAdapter généré utilise un cast typé vers cette interface au lieu d'un cast générique. L'import de l'interface Remote est ajouté automatiquement.

Service source	ServiceAdapter généré
`AccountService implements AccountServiceRemote`	`(AccountServiceRemote) lookupEjb()`
`ClientService` (sans @Remote)	`// TODO: Ajouter le cast vers l'interface Remote`

---

15. Structure du projet généré

Le diagramme ci-dessous illustre l'arborescence complète du projet REST généré.

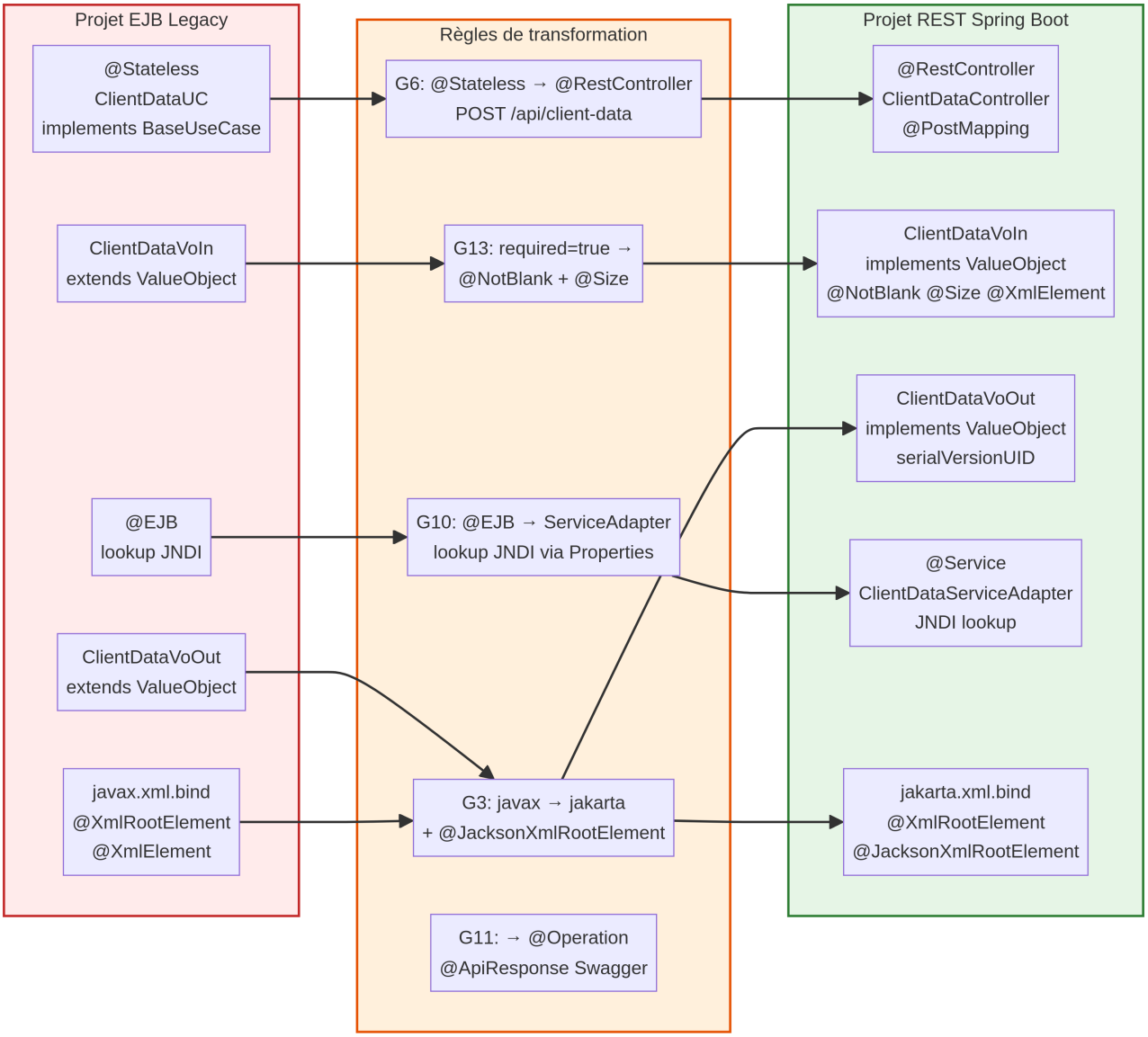


Le projet généré suit la structure standard d'un projet Spring Boot Maven :

```
generated-rest-api/
├── pom.xml                          # Spring Boot 3.2.5, Java 21
├── Dockerfile                       # Multi-stage build (JDK → JRE Alpine)
├── docker-compose.yml               # Orchestration avec health check
├── TRANSFORMATION_SUMMARY.md        # Résumé de la transformation
├── README.md                        # Guide de démarrage
├── src/
│   ├── main/
│   │   ├── java/com/bank/api/
│   │   │   ├── Application.java     # @SpringBootApplication
│   │   │   ├── controller/          # Controllers REST (1 par UseCase/MDB)
│   │   │   ├── service/             # Service Adapters JNDI (1 par UseCase/MDB)
│   │   │   ├── dto/                 # DTOs (VoIn, VoOut)
│   │   │   ├── enums/               # Enums métier recopiés (v2.0)
│   │   │   ├── validation/          # Validateurs custom recopiés (v2.0)
│   │   │   ├── event/               # Events Spring pour MDB (v2.0)
│   │   │   ├── listener/            # EventListeners async pour MDB (v2.0)
│   │   │   ├── ejb/interfaces/      # BaseUseCase, ValueObject (recopiés)
│   │   │   ├── config/              # SecurityConfig, CorsConfig, etc.
│   │   │   ├── exception/           # GlobalExceptionHandler + exceptions custom (v2.0)
│   │   │   ├── filter/              # CorrelationIdFilter
│   │   │   └── logging/             # LoggingAspect
│   │   └── resources/
│   │       ├── application.properties
│   │       ├── application-dev.properties
│   │       └── application-prod.properties
│   └── test/
│       └── java/com/bank/api/controller/ # Tests unitaires (1 par controller)
```

16. Mapping EJB vers REST — Correspondances

Le diagramme ci-dessous illustre les correspondances entre les concepts EJB et les concepts REST Spring Boot.



Le tableau suivant détaille les correspondances élément par élément.

Concept EJB	Concept REST Spring Boot	Règle
`@Stateless`	`@RestController` + `@Service`	G4, G6
`@Stateful`	`@RestController` + commentaire d'avertissement	G4
`@Singleton`	`@RestController` + `@Scope("singleton")`	G4
`@MessageDriven`	Controller async + Event + Listener + ServiceAdapter	v2.0
`BaseUseCase.execute(ValueObject)`	`@PostMapping` + méthode execute	G6
Méthodes publiques multiples	Routes REST individuelles	G5
`@EJB` lookup	`@Service` + JNDI `InitialContext`	G10
`@RolesAllowed`	`@PreAuthorize("hasAnyRole(...)")`	v2.0
`@XmlTransient`	`@JsonIgnore`	v2.0
Enums métier (`@XmlEnum`)	Enums recopiés + `@JsonValue` / `@JsonCreator`	v2.0
Exceptions custom	Recopiées + `GlobalExceptionHandler` enrichi	v2.0
Validateurs (`@Constraint`)	Recopiés avec migration `javax` → `jakarta`	v2.0
Interface `@Remote`	Cast typé dans ServiceAdapter	v2.0
`VoIn extends ValueObject`	DTO `implements ValueObject` + `@NotBlank` + `@Size`	G13
`VoOut extends ValueObject`	DTO `implements ValueObject` + `serialVersionUID`	G1
`javax.xml.bind.*`	`jakarta.xml.bind.*` + `@JacksonXmlRootElement`	G3
`@XmlRootElement`	`@XmlRootElement` + `@JacksonXmlRootElement`	G3, G12
`@XmlElement(required=true)`	`@XmlElement(required=true)` + `@NotBlank`	G13
Exceptions métier	`GlobalExceptionHandler` + `@ControllerAdvice`	G9
Nom JNDI	`@Value("${ejb.jndi...}")` externalisé	G10

---

17. Score de qualité et métriques

17.1 Calcul du score

Le SmartCodeEnhancer calcule un **score de qualité sur 100** à la fin de son exécution. La formule de calcul est la suivante :

> ***Score = min(100, (règles appliquées / règles vérifiées) × 100 + bonus critiques)***

Le **bonus critiques** est calculé comme : `(règles critiques appliquées / règles critiques totales) × 10`. Ce bonus valorise l'application des règles les plus importantes (sécurité, validation, observabilité).

17.2 Niveaux de sévérité

Sévérité	Signification	Exemples
Critical	Règle indispensable pour la sécurité ou le fonctionnement	SecurityConfig, @Valid, CorrelationIdFilter
Warning	Règle importante pour la qualité du code	Codes HTTP, tests unitaires, Actuator
Suggestion	Bonne pratique recommandée	Dockerfile, pagination, compression gzip
Info	Vérification d'audit (pas de modification)	Kebab-case, JSON par défaut

17.3 Rapport d'amélioration

Le rapport d'amélioration (`EnhancementReport`) contient pour chaque règle : l'identifiant (R01 à R76), la catégorie, la sévérité, la description, le fichier impacté, et un indicateur booléen d'application. Il fournit également les compteurs globaux : nombre total de règles vérifiées, nombre de règles appliquées, et score de qualité.

---

18. Annexes techniques

18.1 Technologies utilisées

Composant	Technologie	Version
Application outil	Spring Boot	3.x
Moteur de templates	Thymeleaf	3.x
Analyse statique	JavaParser	3.x
Projet généré	Spring Boot	3.2.5
Java cible	OpenJDK	21
Build	Maven	3.8+
Documentation API	SpringDoc OpenAPI	2.5.0
Sécurité	Spring Security	6.x
Conteneurisation	Docker	Multi-stage

18.2 Prérequis d'utilisation

L'outil nécessite un projet EJB source au format ZIP respectant les conventions suivantes : les classes EJB doivent être annotées avec `@Stateless` , `@Stateful` , `@Singleton` ou `@MessageDriven` ; les DTOs doivent suivre les conventions de nommage `*VoIn` , `*VoOut` , `*Dto` ou `*DTO` ; le projet doit contenir des fichiers `.java` compilables.

18.3 Limitations connues

L'outil ne reproduit pas l'état conversationnel des EJB `@Stateful` . Les Service Adapters effectuent un lookup JNDI à chaque appel (pas de cache par défaut). Les tests unitaires générés mockent les Service Adapters et ne testent pas l'intégration réelle avec le serveur EJB. Les MDB transformés en composants event-driven nécessitent la configuration d'un `TaskExecutor` Spring pour le traitement asynchrone.

18.4 Historique des versions

Version	Date	Changements
1.0	Mars 2026	Version initiale : transformation EJB → REST, 14 règles G, 77 règles SmartCodeEnhancer
2.0	Mars 2026	Transformation MDB event-driven, extraction enums/exceptions/validateurs, commentaires de traçabilité, @RolesAllowed → @PreAuthorize, @XmlTransient → @JsonIgnore, interface @Remote typée